

HW3: Cache Optimization

Lin Yen-Yu
112550098

Abstract—This work investigates performance and hardware trade-offs in the Aquila L1 data cache by varying associativity, replacement policies, and cache capacity. A profiling mechanism was implemented to measure hit rates, miss types, and latency across configurations. Results show that 4 KB caches gain little from higher associativity or LRU-based policies, while expanding to 8 KB—especially with 2-way FIFO—provides the best improvement with minimal hardware cost. Additional attempts such as prefetching and write buffering were explored but remained incomplete due to system-level interactions in the cache control FSM.

Index Terms—RISC-V, data cache, associativity, replacement policy, FPGA synthesis, write-back cache, prefetching, write buffer

I. INTRODUCTION

Cache performance plays a critical role in the overall efficiency of the Aquila RISC-V processor, especially when running memory-intensive workloads such as CoreMark. The baseline Aquila design uses a 4 KB, 4-way, FIFO-based L1 data cache. While simple and hardware-efficient, its behavior under different associativity levels, replacement policies, and cache sizes has not been fully explored.

In this homework, I extend the baseline D-cache in three directions: (1) modifying the number of ways, (2) implementing alternative replacement policies including LRU and PLRU, and (3) increasing the capacity from 4 KB to 8 KB. To quantify the effects, I built a cycle-level profiling mechanism inside the software runtime to measure read/write hits, clean/dirty misses, and average memory latency. The results, along with synthesis and power reports, provide insight into the trade-offs between performance and hardware cost. Additionally, I experimented with prefetching and write buffer mechanisms to further improve miss behavior.

II. CACHE PROFILING MECHANISM

All profiling is performed inside the `main` function. Similar to previous assignments, a start and end flag control the `is_window` signal so that only the execution of `main` is measured while initialization code (e.g., `crt0`) is excluded. The correct insertion points for these flags were determined using `objdump`.

Read/write classification is obtained directly from the bus signal `p_rw_i`, following the same logic as the D-cache. Hit and miss outcomes come from the internal signal `cache_hit`. Clean and dirty misses are distinguished by checking whether the chosen victim line has its dirty bit set.

Latency is measured using `lat_current`, counting cycles from when a request is issued (`p_strobe_i`) until memory responds (`m_ready_i`). Statistics are collected for read/write

hits, read/write misses, and clean/dirty misses, providing a complete view of cache access behavior.

III. D-CACHE OPTIMIZATION

In this homework, I explored three different directions for optimizing the D-cache design: changing the associativity, modifying the replacement policy, and adjusting the overall cache capacity.

A. Changing the Number of Ways

The original design uses a 4-way set-associative cache. I experimented with both 2-way and 8-way configurations. Besides updating the `N_way` parameter, I also needed to modify how `way_hit` and `cache_hit` are computed.

Other parts of the design, such as `hit_index` and `FIFO_cnt`, were already fully parameterized in the baseline, so they worked naturally without further changes.

B. Changing the Replacement Policy

The baseline Aquila D-cache uses a FIFO replacement policy. For comparison, I implemented two additional schemes, giving me a total of three policies:

- 1) **FIFO**: The simplest design. Each set maintains a circular pointer (the FIFO counter), and the oldest way is replaced.
- 2) **LRU (Least Recently Used)**: This policy evicts the way that has not been used for the longest time. To support this, every **line in every way** needs an `LRU_cnt` field to track recency. The counters must be updated **on both hits and misses**. This makes pure LRU noticeably more expensive in hardware than FIFO.

- 3) **PLRU (Pseudo-LRU)**: To reduce the cost of true LRU, I adopted a binary-tree based PLRU structure. Instead of storing a full counter per way, each **line** has a small tree of direction bits that indicate which subtree contains the least-recently-used way.

The PLRU tree is updated using the accessed way index `accessed_way[WAY_BITS-1:0]`. Each access flips the corresponding tree bits so that the pointer eventually points to the next victim.

For example, in the tree above:

- the bit pattern (0,0,0) corresponds to Way-0,
- the bit pattern (1,0,0) corresponds to Way-2.

Compared to LRU:

- PLRU only needs to maintain a few direction bits per line,
- updates are simpler and cheaper,
- behavior closely approximates true LRU.

	FIFO			LRU			PLRU		
	4KB			4KB			4KB		
	2WAY	4WAY	8WAY	2WAY	4WAY	8WAY	2WAY	4WAY	8WAY
total_speed (%)	100.00	100.00	100.00	100.00	100.81	100.83	100.00	100.00	100.00
read hit (%)	77.59	77.59	77.59	77.59	76.88	76.88	77.59	77.59	77.59
read dirty miss (%)	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
avg read latency (clk)	53.37	53.36	53.36	53.37	53.36	53.34	53.37	53.37	53.37
write hit (%)	83.30	83.30	83.30	83.30	83.36	83.36	83.30	83.30	83.30
write dirty miss (%)	99.99	99.99	99.99	99.99	100.00	100.00	99.99	99.99	99.99
avg write latency (clk)	53.64	53.65	53.65	53.65	53.54	53.63	53.65	53.64	53.65
LUTs	1712	2579	2985	2198	3210	4103	2198	2644	3046
LUT (logic)	1696	2563	2769	2182	3194	3887	2182	2628	2830
LUT (mem)	16	16	216	16	16	216	16	16	216
D Flip-Flop	1136	1136	1296	1263	1518	1959	1263	1200	1417
BRAM	5	10	16	5	10	16	5	10	16
Power (W)	0.961	0.980	0.997	0.957	0.979	1.007	0.957	0.975	0.987

TABLE I: D-cache performance under different associativity and replacement policies.

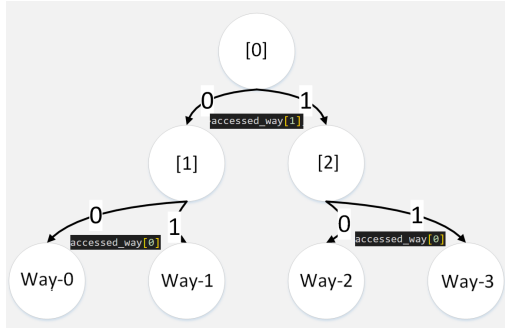


Fig. 1: Example binary-tree PLRU structure (4-way)

C. Changing the Cache Capacity

Finally, I modified the cache capacity by adjusting `define DCACHE_SIZE` in `aquila_config`. I compared the baseline 4 KB cache to an 8 KB version to evaluate how capacity affects performance.

IV. ANALYSIS RESULT

All performance metrics are measured relative to the baseline **4KB, 4-way, FIFO** D-cache. The total speed is normalized to the baseline execution time. The read/write hit rate is computed by dividing the number of hits by the total number of reads/writes, respectively. The average read/write latency is obtained by dividing the total accumulated latency by the number of read/write requests.

Across all configurations, I observed that almost all misses are **dirty misses**, while clean misses rarely occur. Moreover, both read and write latency remain close to **53 cycles**, indicating that external memory latency dominates the overall access time.

A. Changing N_{ways} and Replacement Policy

1) *Performance Analysis*: Based on the results in Table 1 (4KB capacity), changing either the associativity (2-way, 4-way, 8-way) or the replacement policy (FIFO, LRU, PLRU) does not lead to any significant improvement in total speed, hit rate, or average latency.

This is primarily because the CoreMark workload has a relatively small working set, resulting in very few conflict misses. Therefore, increasing associativity or using a more advanced replacement policy cannot meaningfully improve cache behavior. In addition, the DRAM access latency dominates the overall performance, masking small differences in cache-level behavior.

2) *Hardware Resource Analysis*: From Table 2 (4KB), we can see that:

- **FIFO 2-way** consumes the smallest number of LUTs and also has low flip-flop and power usage.
- **LRU 2-way** achieves the lowest power consumption, although its LUT and FF usage are higher than FIFO.

Therefore, if the hardware budget is limited and the capacity must remain at 4KB, **FIFO 2-way is the most cost-efficient choice**, while **LRU 2-way** can be considered if power consumption is prioritized.

B. Changing D-cache Capacity

	FIFO			LRU		
	8KB			8KB		
	2WAY	4WAY	8WAY	2WAY	4WAY	8WAY
total_speed (%)	83.73	95.39	97.91	86.99	99.65	99.75
read hit (%)	83.17	80.60	79.23	83.17	77.73	77.73
read dirty miss (%)	100.00	100.00	100.00	100.00	100.00	100.00
avg read lat (clk)	53.34	53.37	53.35	43.44	53.34	53.37
write hit (%)	93.36	84.17	83.30	89.78	83.45	83.38
write dirty miss (%)	99.99	99.99	99.99	99.99	99.99	100.00
avg write lat (clk)	53.52	53.56	53.64	53.44	53.50	53.60
LUTs	1971	2759	2862	2518	4096	4861
LUT (logic)	1939	2727	2830	2486	4064	4829
LUT (mem)	32	32	32	32	32	32
D Flip-Flop	1266	1272	1210	1521	2033	2546
BRAM	5	10	20	5	10	20
Power (W)	0.94	0.98	1.00	0.96	0.99	1.03

TABLE II: D-cache performance under different associativity for 8KB capacity (FIFO vs LRU).

1) *Performance Analysis*: When increasing the capacity to 8KB (Table), the performance impact becomes much more noticeable. FIFO benefits the most:

- **FIFO 2-way** improves total speed to only **83.73%** of the baseline execution time.

This acceleration primarily comes from a significant increase in the write hit rate, which reduces the number of expensive dirty write misses.

LRU also benefits, but for a different reason:

- **LRU 2-way** reaches **86.99%** of baseline runtime.

While write hit rate slightly improves, the dominant effect is that the **average read latency decreases by nearly 10 cycles**, meaning that LRU more effectively prevents read conflict misses when the cache has larger capacity.

2) *Hardware Resource Analysis*: From Table 2, the following trends can be observed:

- **LRU 8-way** consumes the most LUTs, flip-flops, and power.
- **FIFO 2-way** consistently uses the least amount of hardware resources across LUTs, FFs, and power.

Compared to the 4KB configurations, the 8KB versions require more LUTs and FFs due to the increased number of sets and lines. However, BRAM usage increases significantly only for the 8-way configurations.

Combining both performance and hardware considerations, we conclude that:

FIFO 8KB 2-way is the best overall design:
minimal hardware cost and the largest performance gain.

V. DISCUSSION

Name	Constraints	Status	WNS	TNS
✓ synth_1 (active)	constrs_1	synth_design Complete!		
✓ impl_1	constrs_1	write_bitstream Complete!	-0.560	-24.043

Fig. 2: LRU 8kb 8way with negative WNS

During synthesis of the **LRU, 8KB, 8-way** configuration, I observed a negative WNS (Worst Negative Slack), indicating that the design fails to meet timing. This result is reasonable because this configuration requires **4861 LUTs**, the largest among all evaluated designs.

True LRU is hardware-intensive: each access must update multiple counters, and an 8-way cache requires eight tag comparisons and a large victim-selection network. With the increased capacity (8KB), the critical path becomes deeper and routing congestion worsens, making timing closure difficult on the Aquila FPGA.

Thus, the **LRU 8KB 8-way** design is not practical. Its high hardware cost and inability to satisfy timing constraints make it unsuitable for real implementation.

VI. IMPROVEMENTS I HAVE TRIED

A. Prefetching

I attempted to implement a simple prefetching mechanism. After a read miss, when the cache fetches the requested line from memory, I also tried to fetch the *next* cache line. This should help reduce consecutive read misses for sequential memory accesses.

To support prefetching, I modified the *RdfromMem* logic, added a *prefetch_phase* register to track whether the

current read is the original line or the prefetched one, and updated related logic such as *FIFO_cnt* and the data write path.

However, after increasing the Pi-digit workload to 500 and repeatedly testing with simulation, I was unable to complete a fully working design. The main issue was that the processor stopped asserting *p_strobe_i* during *sw* instructions, and I could not identify the root cause. Since my prefetch logic was embedded inside *RdfromMem*, I suspect a cleaner approach would be to introduce a dedicated *prefetch state* in the FSM instead of overloading the existing read-miss flow.

```

always @(posedge clk_i)
begin
    if (rst_i) begin
        prefetch_phase <= 1'b0;
    end
    else if (S == RdfromMem && m_ready_i && !rw && !prefetch_phase) begin
        // after fetching original line, enter prefetch phase
        prefetch_phase <= 1'b1;
    end
    else if ((S == RdfromMem && m_ready_i && !rw && prefetch_phase) || S == Idle) begin
        // reset when prefetch completes or Idle
        prefetch_phase <= 1'b0;
    end
end

```

Fig. 3: prefetching_phase logic in *RdfromMem* state

B. Write Buffer

After the prefetching attempt failed, I also explored adding a write buffer. The idea is that on a write miss, the data is inserted into the buffer and the D-cache immediately returns *ready*, allowing the processor to continue. The cache then writes the buffered entries back to memory in the background, which can reduce the visible write-back latency.

I added a *Wbflush* state and allowed the buffer to drain whenever the D-cache is *IDLE*. Several coherence issues must be handled, including:

- returning buffered data for reads targeting addresses still in the write buffer,
- merging or ordering multiple writes to the same address,
- ensuring consistency between dirty evictions and buffered writes.

However, the design did not function correctly. During simulation, the processor stopped asserting *p_strobe_i* on *sw* instructions (e.g., at 0x800018f8), preventing write requests from entering the cache. I was unable to identify the cause, but I suspect an interaction between the new write-buffer logic and existing D-cache or UART initialization states.

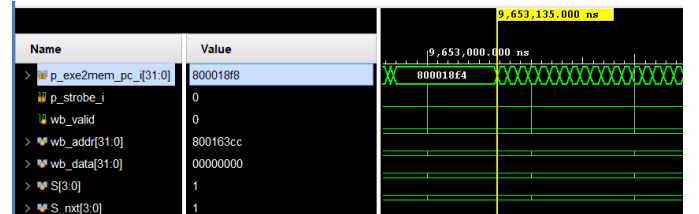


Fig. 4: write_buffer simulation

REFERENCES

- [1] <https://zhuanlan.zhihu.com/p/10063787407>
- [2] https://people.cs.nycu.edu.tw/~ttyeh/course/2025_Spring/CS10014/outline.html